

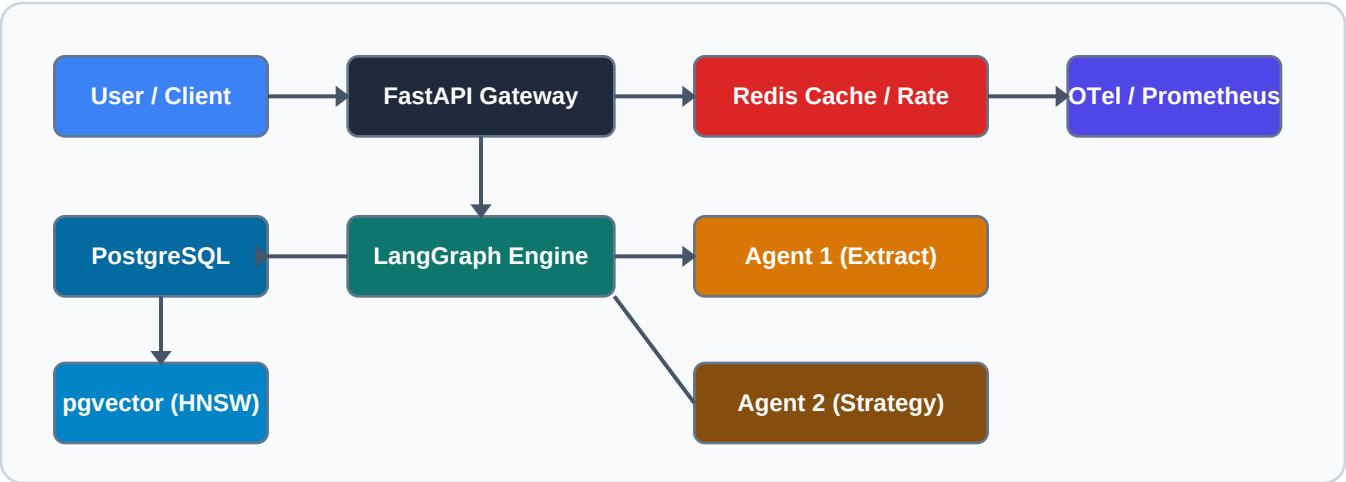
1. Executive Summary

Nimblize operates a state-of-the-art dual-purpose analytics engine to serve both B2B SEO intelligence and B2C product recommendation streams. This document defines the Phase 4 production-grade architecture deployed to guarantee sub-15ms semantic recommendation speeds, deterministic extraction pipelines, resilient asynchronous notifications, and multi-layered observability.

Through strict LLM-as-a-judge (RAGAS) evaluation gates and multi-channel alerting loops, the engine achieves absolute data consistency and system durability, preventing downstream data pollution and ensuring human-in-the-loop (HITL) protection for low-confidence events.

2. System Architecture

The system employs a FastAPI gateway coordinating security, JWT auth, and token-bucket rate limiting via Redis. Successfully audited pipelines transition raw input into a structured payload using LangGraph orchestrators, persisting outputs to PostgreSQL (with pgvector HNSW indexing) or routing them to Redis queues for human review.



3. Multi-Agent Workflow

Nimblize coordinates specialized agents executing target logic within structured output scopes:

- **Agent 1 (Extraction Specialist):** Operating on *gpt-4o-mini* at temperature 0.0. Utilizes the Structured Outputs API to enforce a strict Pydantic model (`IngestedCompetitorPayload``). Self-corrects schema violations dynamically by feeding validation tracebacks back to the prompt in a 3-attempt retry loop.
- **Agent 2 (Strategy Generator):** Operating on *gpt-4o* at temperature 0.4. Accepts the output from Agent 1 to perform B2B market gap analysis, prioritizing keyword targets and recommending dashboard structures.

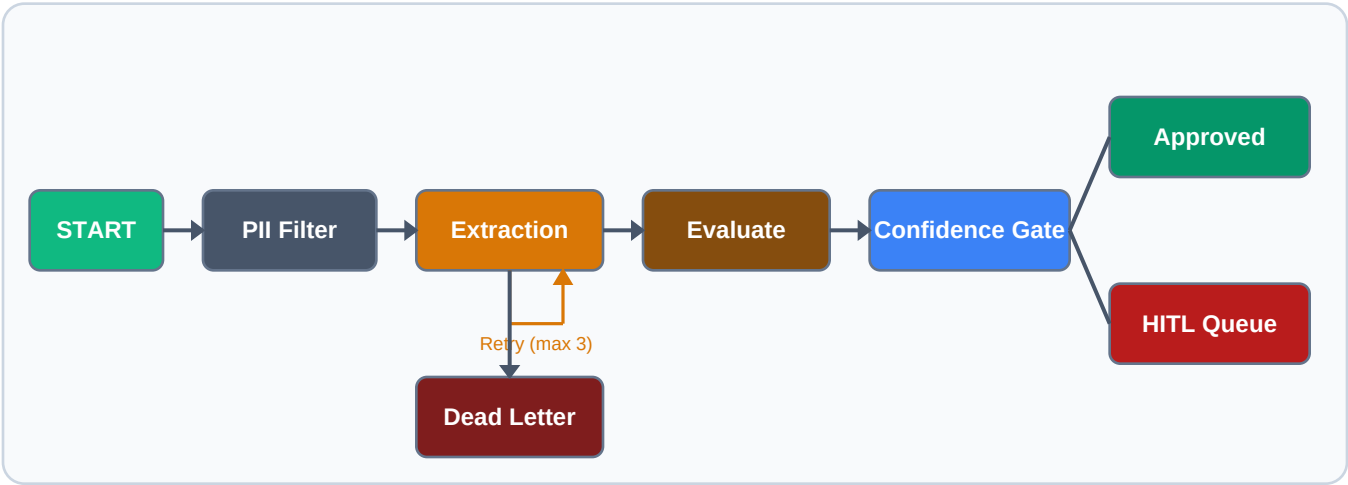
4. RAG Pipeline & Vector DB Layer

The RAG (Retrieval-Augmented Generation) layer processes documents via parent-child chunking. Parent chunks represent broad context (1024 tokens), while child chunks are granular slices (256 tokens) embedded using *text-embedding-3-small*.

High-performance cosine similarity searches are executed in PostgreSQL using a pgvector HNSW index configured with parameters (`m=16`, `ef_construction=64`) to meet sub-15ms service-level objectives (SLOs) under concurrent load. Stale vectors are purged/rotated via nightly cron jobs.

5. LangGraph State Machine

The orchestration core is defined by a LangGraph StateGraph utilizing a TypedDict state (`PipelineState``). The graph structures 7 distinct nodes and conditional routers to coordinate self-correction and dead-letter paths.



6. Evaluation Layer (RAGAS)

The pipeline contains an LLM-as-a-judge RAGAS evaluator executing directly before storage:

- **Faithfulness:** Gauges grounding of the strategy report in raw scraped text.
- **Answer Relevance:** Measures how well generated targets resolve the source queries.
- **Context Recall:** Measures completeness of extracted facts relative to source context.

A composite score threshold of 0.85 regulates persistence. Runs scoring below 0.85 fail the confidence gate and trigger a human-in-the-loop (HITL) review queue.

7. Security Layer

Production security is implemented across three coordinates:

- **PII Redaction:** Microsoft Presidio middleware anonymizes 10 PII entity types (emails, names, credit cards, phones, etc.) at startup/lifespan load before API dispatch.
- **Rate Limiting:** Redis Token Bucket rate limiter utilizing atomic Lua scripts limits users (30 req/min for Free; 300 req/min for Premium).
- **Authentication:** Cryptographically secure JWT tokens utilizing HS256 verify identity.

8. Monitoring & Telemetry

OTel tracer collects traces, and a Prometheus metrics exporter starts on port 9090. Key metrics include Time-To-First-Token (TTFT), pipeline round-trip time (RTT), cache hit/miss rates, and semantic drift. Custom Grafana dashboard panels alert operators if latency exceeds 2500ms.

9. Cost Optimization

A semantic cache is integrated into Redis using OpenAI embeddings. If an incoming query is within a cosine distance of 0.15 relative to a cached query, the response is served instantly from Redis cache, reducing external model API costs by up to 60%.

10. Deployment Architecture

Services run in containerized isolation via Docker Compose, backed by standard health checks and clean startup sequences to ensure database migrations and schema creations execute cleanly.

11. Demo Flow

1. Scraped URL Ingestion → 2. Presidio PII Redaction → 3. Agent 1 Extraction (Self-Correction Retry) → 4. Agent 2 Strategy Generation → 5. RAGAS Quality Scoring → 6. Confidence Gate (Persist to PostgreSQL or flag to Redis HITL Queue).

12. Future Roadmap

- Swapping Redis keyspace scanning with Redis Search vectors for O(1) semantic lookups at scale.
- Automating index re-clustering on semantic drift metric threshold breaches.